חלק א

.1

```
Solution
O(n^2)
```

.2

```
Solution
def breakIt(f, x):
    while x
        yield x
        x = f(X)
```

.3

```
Solution
O(nlen(lst))
```

.4

```
Solution
def tree_cut(t,k):
    t.branches = [tree_cut(b, k) for b in t.branches[:k]]
    return t
```

.5

```
Solution
def get_min_tree(t):
    new_bran = [get_min_tree(b) for b in t.branches]
    return Tree(min([b.label for b in new_bran] + [t.label]), new_bran)
```

.6

Solution

```
def gen(t, n):
    for b in t.branches:
        if t.label + b.label == n:
            yield [t.label, b.label]
        yield from gen(b, n)
```

.7

Solution

```
def sub_lists(lst):
    if not lst:
        return [[]] #..<1>..
    else:
        sub = sub_lists(lst[1:]) # ..<2>..
        return inject_each(lst[0], sub) + sub # ..<3>..
```

חלק ב׳

.8

Solution

```
def insert_average(t, val):
    if t.branches:
        total_labels = sum(branch.label for branch in t.branches)
        avg_val = round((total_labels + val) / (len(t.branches) + 1), 2)
        new_branches = [insert_average(branch, val) for branch in t.branches]
        new_branches.append(Tree(avg_val))
    else:
        new_branches = []
    return Tree(t.label, new_branches)
```

.9

Solution

```
def long_ways(t):
    def g(s):
        if t.label > s.label :
            return 0
        elif not s.branches:
            return 1
        else:
            return sum([g(br) for br in s.branches])
    return g(t)
```

.10

```
Solution
def inter_link_list(lnk, lst):

    def insert_node(ev_od, curr):
        if ev_od:
            point = Link(ev_od.pop(0), curr.next)
            curr.next = point
            return point
        return curr

    even,odd=[i for i in lst if i % 2 == 0],[i for i in lst if i % 2 == 1]
    dummy = curr = Link(0, lnk)

    while curr and curr.next:
        if curr.value % 2 == curr.next.value % 2:
            curr = insert_node(odd if curr.value % 2 == 0 else even, curr)
        curr = curr.next
    return dummy.next
```

.11

```
Solution
def fly_filter(iter1, iter2):
    curr_pos = 0
    while True:
        try:
            next_pos = next(iter1)
            to_skip = next_pos - curr_pos
            while to_skip > 0:
                next(iter2)
                to_skip -= 1
                curr_pos += 1
            yield next(iter2)
            curr_pos += 1
        except StopIteration:
            break
```

.12

```
Solution
class Eyes:

    def __init__(self):
        self.seen = []

    def spy(self, it1):
        self.seen.extend(it1)
```

```
    def record(self, it2):
        count_seen = {item: 0 for item in it2}
        for item in self.seen:
            if item in count_seen:
                count_seen[item] += 1
        return count_seen


    def free(self, it3):
        it3_set = set(it3)
        self.seen = [item for item in self.seen if item not in it3_set]

    def __repr__(self):
        return str(self.seen)
```

.13

```
Solution
def periMatrix(n, k):
    matrix = [[0]*n for _ in range(n)]
    count = 1

    layer = k-1  # Layer to write the numbers in

    # the top row
    for i in range(layer, n-layer):
        matrix[layer][i] = count
        count += 1

    # the right column
    for i in range(layer+1, n-layer):
        matrix[i][n-layer-1] = count
        count += 1

    # the bottom row
    for i in range(n-layer-2, layer-1, -1):
        matrix[n-layer-1][i] = count
        count += 1

    # the left column
    for i in range(n-layer-2, layer, -1):
        matrix[i][layer] = count
        count += 1

    return matrix
```

כדי להדפיס את המטריצה אפשר להשתמש בפונקציה הבאה :

```
def print_mat(matrix):
```

```
    for row in matrix:
        print(" ".join(str(num).rjust(2) for num in row))

print_mat(periMatrix(n, k))
```